

Ex91_LMS_linear_predictor

May 3, 2026

1 MSE StatDig : Chap 9 “Adaptive filtering”

1.1 Ex 9.1 LMS linear predictor

ver : DLY/04.05.2026

1.2 General

1.2.1 Description

Let ($x[n]$) be a second-order autoregressive process AR(2) that is generated according to the difference equation:

$$x[n] = -a[1] \cdot x[n-1] - a[2] \cdot x[n-2] + v[n] \quad = 1.2728 \cdot x[n-1] - 0.81 \cdot x[n-2] + v[n]$$

Where ($v[n]$) has unit variance. So the coefficients are:

$$a[1] = -1.2728a[2] = 0.81$$

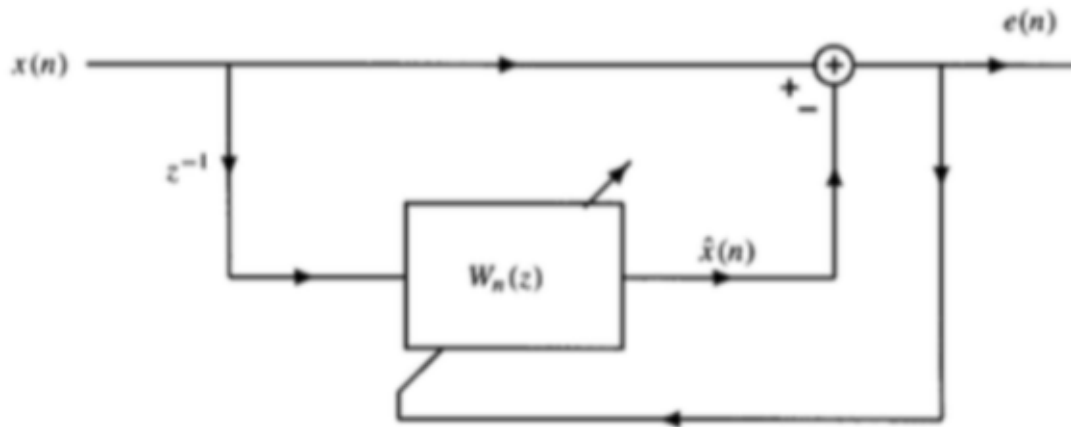
It gives a transfer function:

$$\frac{X(z)}{V(z)} = \frac{1}{1 + a[1]z^{-1} + a[2]z^{-2}} = \frac{1}{1 - 1.2728z^{-1} + 0.81z^{-2}}$$

The goal is to design a one-step predictor using the adaptive LMS and NLMS algorithms and to compare it to a Wiener filter.

This comparison to Wiener is possible because the process is stationary.

Below is the scheme of the one-step adaptive predictor.



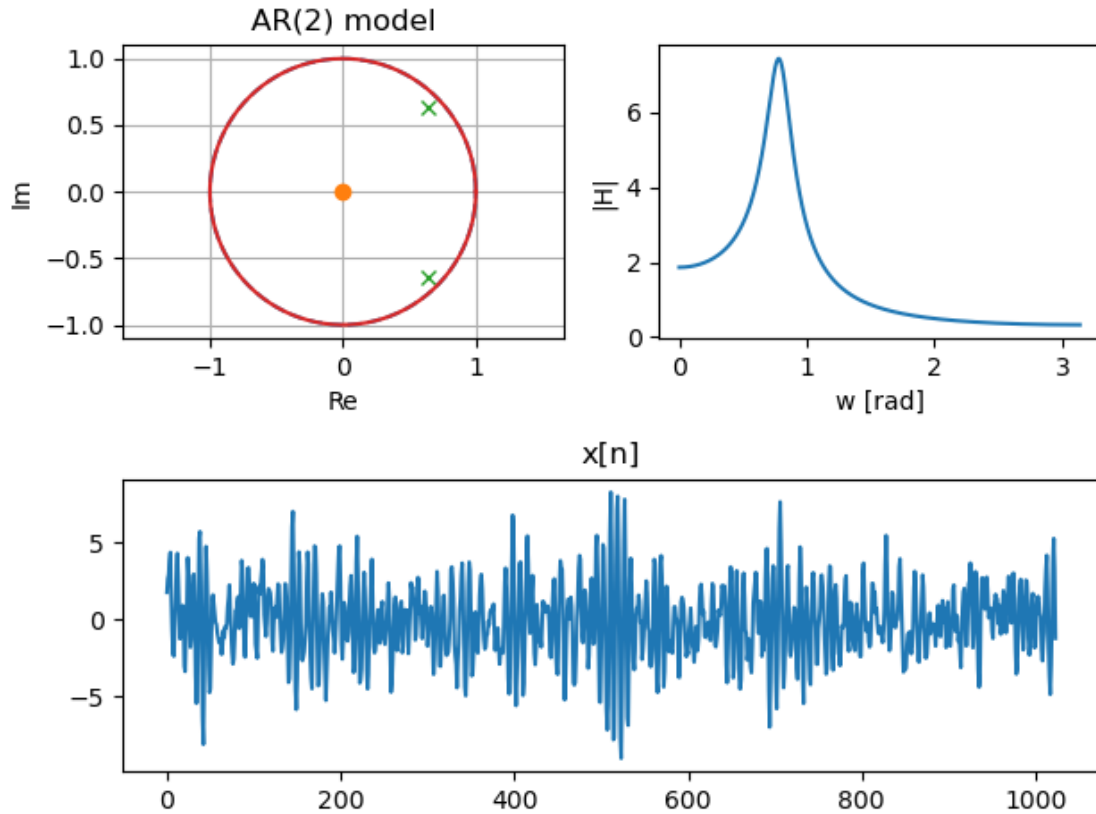
1.2.2 Work

Ex1 : Signal generation

- Display the signals created by the following code :

```
# parameters of the AR(2) process
b = 1;
a = [1,-1.2728,0.81];
N = 1024;
SigmaW = 1

# fix the random seed for reproducibility
np.random.seed(0)
wn = SigmaW*np.random.normal(0,1,N) # noise process
x = signal.lfilter(b,a,wn) #AR(2) process
```



Ex2 : Wiener-Hopf As it is a WSS, the coefficients can be found with the Wiener-Hopf equations. The Wiener filter is designed with the estimation done on the data. The optimum causal linear predictor for $x[n]$ is:

$$\hat{x}[n] = w[1]x[n-1] + w[2]x[n-2]$$

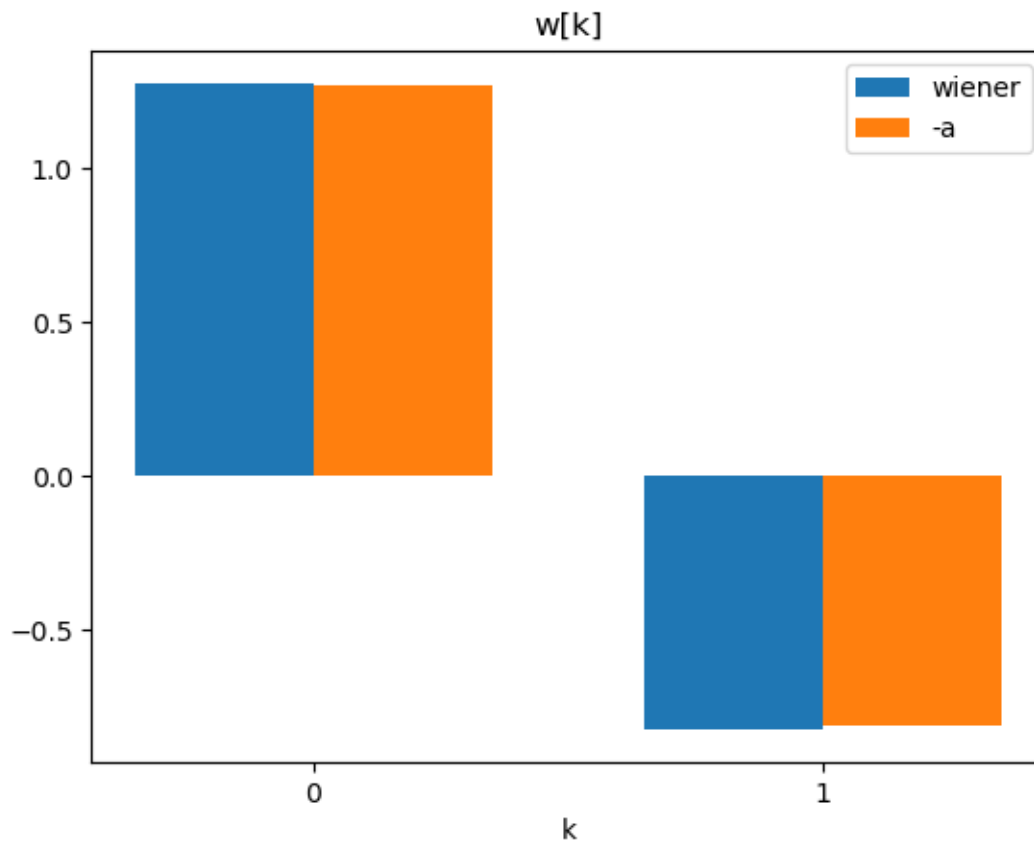
$$\hat{x}[n] = 1.2728x[n-1] - 0.81x[n-2]$$

Ex2.1 Design filter

- Design the prediction filter 1 step using the Wiener-Hopf equations ($p=2$). You will observe that the coefficients are very close to the AR(2) coefficients.:

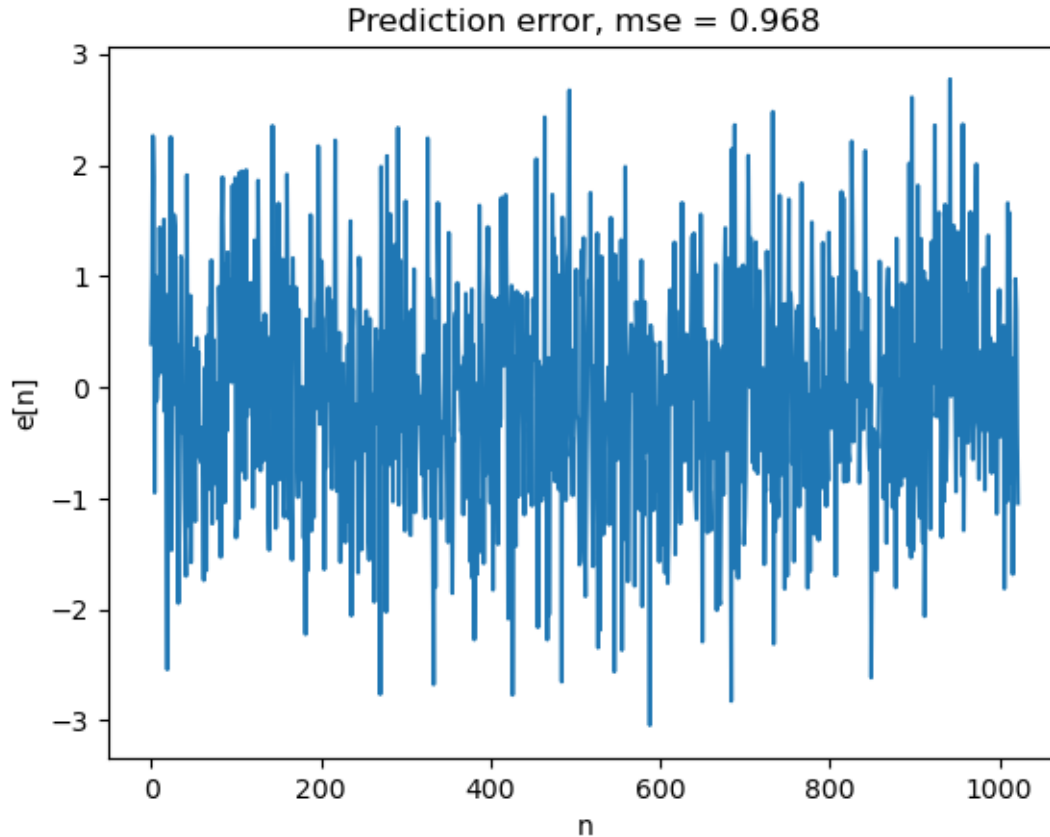
$w = [1.27848692 \ -0.8236305]$

$E = 0.989$



Ex2.2 Prediction

- compute the prediction error $e[n] = x[n] - \hat{x}[n]$ and plot the prediction error with the MSE.



Ex3 : LMS The predictive parameters can be also found using the LMS algorithm.

With the LMS algorithm, the predictor coefficients $w_n[k]$ are updated as follows:

$$w_{n+1}[k] = w_n[k] + \mu e[n] x^*[n - k]$$

Ex3.1 Design filter For the design, we will compare two different rates $\mu : 5 \cdot 10^{-4}$ and $1 \cdot 10^{-2}$.

- Implement the function `myLMS` to compute the coefficients across time

```
def myLMS(x, d, mu, p):
    """
    myLMS Adaptive filtering using the Widrow-Hoff LMS algorithm.

    Parameters:
        x      : 1D array, input data to the adaptive filter
        d      : 1D array, desired output
        mu     : float, LMS step-size (update) parameter
        p      : int, number of filter coefficients

    Returns:
```

```

    W : 2D array of shape (N, nord), filter coefficients over time
    e : 1D array of shape (N,), error signal over time
"""
# your code

return W, e

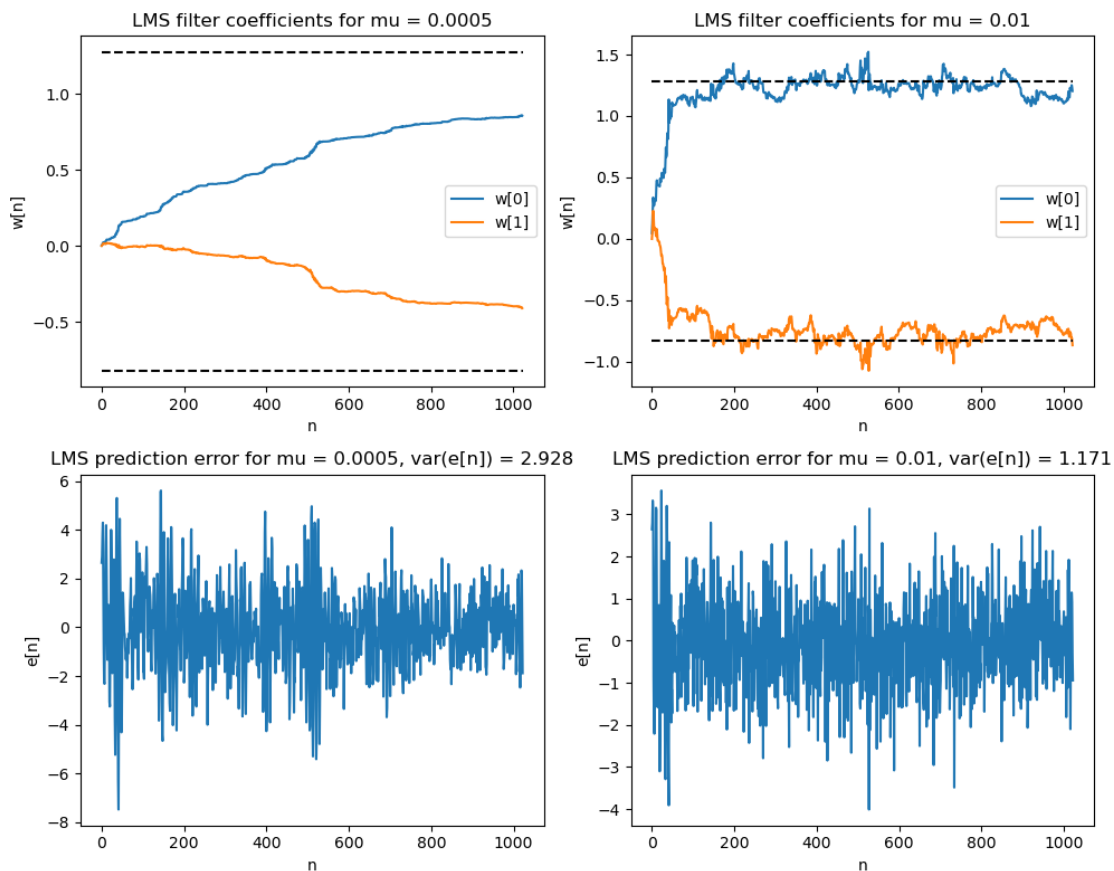
```

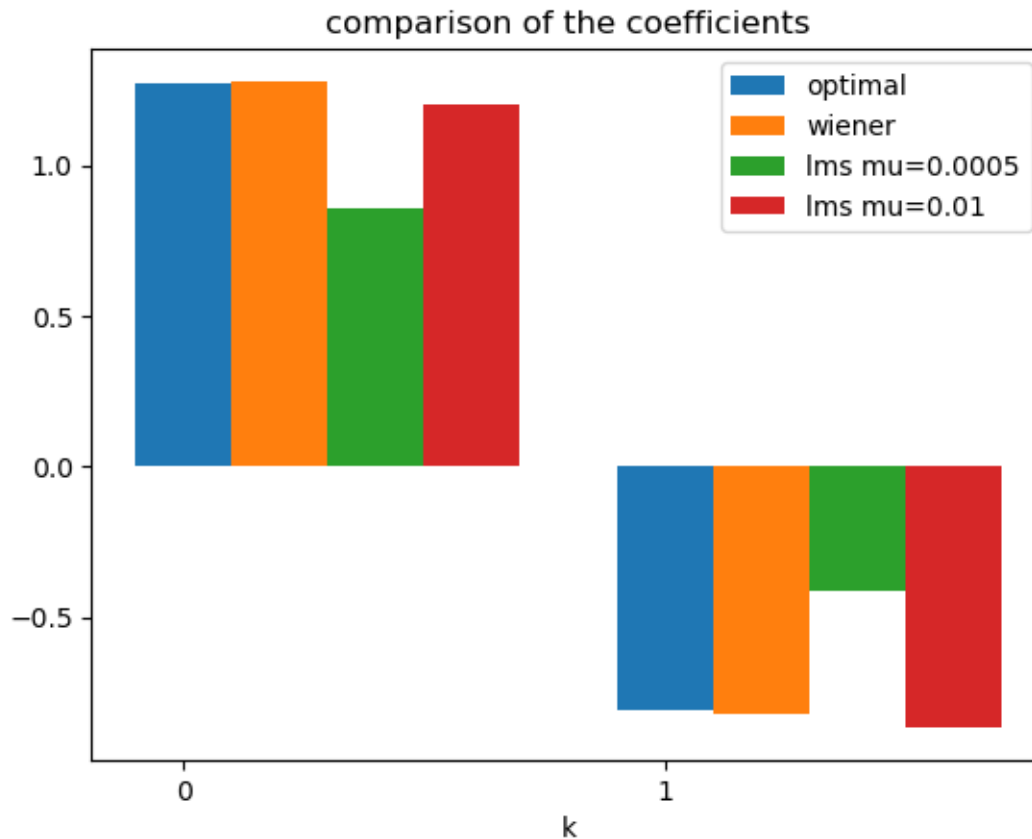
- Plot the coefficients of the filter across time for the two values of μ and compare them to the Wiener filter.
- Display in a bar plot the optimal coefficients, the coefficients obtained with wiener and the last obtained with LMS for the two values of μ .

```

[[1. 0. 0.]
 [2. 1. 0.]
 [3. 2. 1.]
 [4. 3. 2.]
 [5. 4. 3.]
 [0. 5. 4.]
 [0. 0. 5.]]

```





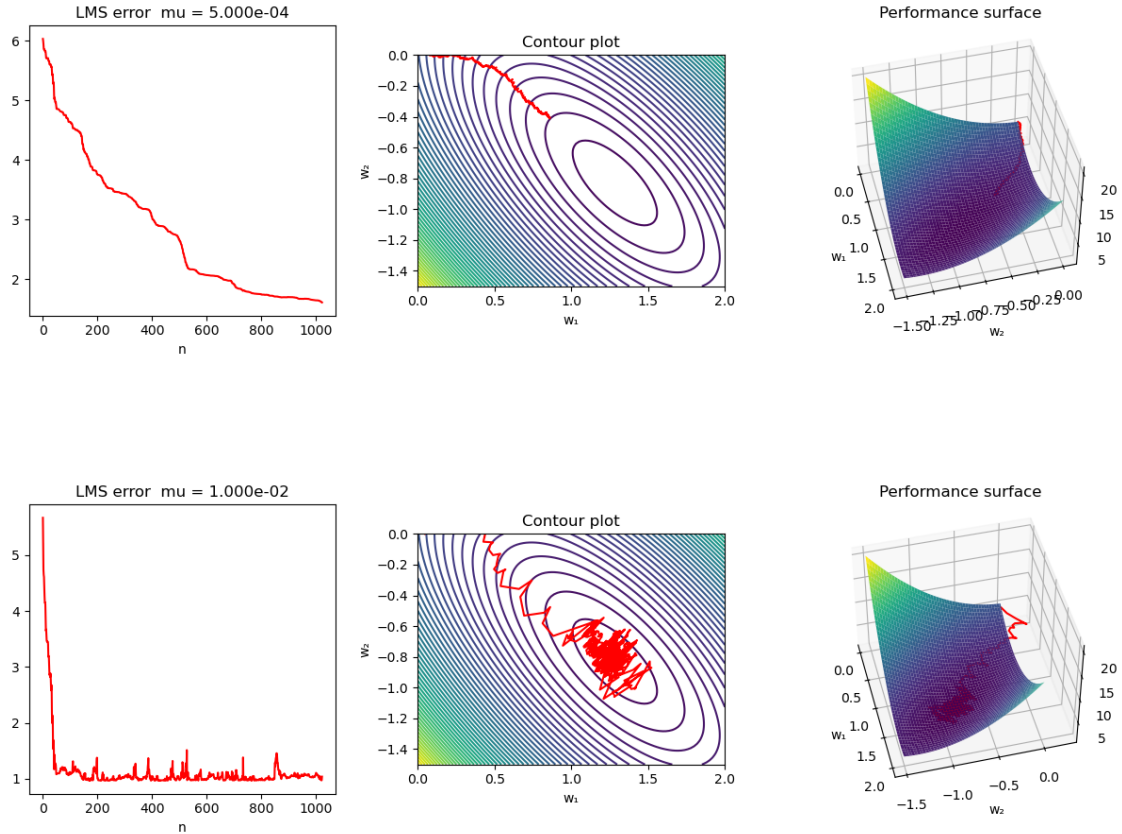
Ex3.2 Performance Surface Finally, a performance surface is drawn with the two rates. It is showing the path realized on the error surface by the optimizer.

- Implement the function `compute_prediction_mse` to compute the performance surface and plot it.

```
def compute_prediction_mse(x,w):
    """
    Computes the prediction MSE for a given filter w and input signal x.

    Parameters:
        x : 1D array, input signal
        w : 1D array, filter coefficients

    Returns:
        mse : float, mean square error of the prediction
    """
    # your code
    return mse
```



Ex4 : NLMS The predictive parameters can be also found using the normalized LMS algorithm also called NLMS.

With the NLMS algorithm, the predictor coefficients $w_n[k]$ are updated as follows:

$$w_{n+1}[k] = w_n[k] + \beta \frac{x^*[n-k]}{\varepsilon + x^2[n-1] + x^2[n-2]} e[n]$$

Ex4.1 Design filter

- Implement the function `myNLMS` to compute the coefficients accross time
- Compute the coefficients with the NLMS algorithm and plot them for $\beta = 0.02$ and $\beta = 0.005$ and with $\varepsilon = 1e-4$
- Compare with the coefficients obtained with the Wiener filter and the LMS algorithm

def `myNLMS(x, d, beta, p, epsilon=1e-6):`

"""

myNLMS Adaptive filtering using the normalized LMS algorithm.

Parameters:

x : 1D array, input data to the adaptive filter

d : 1D array, desired output


```

    beta : float, NLMS step-size (update) parameter
    p : int, number of filter coefficients
    epsilon : float, small constant to avoid division by zero

```

Returns:

```

    W : 2D array of shape (N, norder), filter coefficients over time
    e : 1D array of shape (N,), error signal over time

```

```

"""

```

```

# your code

```

```

return W, e

```

